

PDblend: 性能刻画、在线调度与自适应池配置

PDblend 在同一 GPU 集群中组合混合执行 (M) 与预填充/解码分离 (P/D), 在首 token 延迟 (TTFT) 和平均 token 间隔 (TPOT) 约束下降低服务能耗。一个 instance 是一个推理引擎及其张量并行 (TP) GPU 组; M 在同一实例内完成两个阶段, PD 则使用一对 P、D 实例。方法由三个环节组成: 离线 profiler 提供性能与能耗依据, 请求到达时确定执行路径, 运行期间调整频率、角色及异构 TP 池的流量权重。以下描述当前工作树中的机制, 并在必要处注明不同执行入口的边界。

1 Profiler: 构建可查询的性能与功耗模型

Profiler 的任务是回答: 给定模型、TP 布局、频率和请求形状, 一个实例能以多大代价承担 P、D 或 M 负载。不同 TP 的通信量、权重占用和 KV 容量不同, 因此每个拓扑使用独立 profile; 频率也只在相应测量域内比较。模型版本、引擎、硬件和测量文件共同确定 profile 身份, 避免规划器混用来源不同的性能数据。

采集维度与执行阶段。 P 阶段以输入长度 L 和 GPU 频率 f 为主要维度; D 阶段进一步考虑 batch 大小 B 与上下文长度 C , 逐步解码并记录实际执行形状。M 阶段让 prefill 与 decode 共存, 以观测长 prompt 对已有生成请求造成的停顿。采集先完成加载、预热和时钟设置, 再分别记录时间与功率窗口, 并保留原始记录和独立验证样本。除计算阶段外, 还采集空闲、停车、关闭状态的功率, 以及唤醒、调频、KV 容量与 P/D 交接信息。

从测量到代价函数。 记 $T_p(L, f)$ 为 prefill 时间, $T_d(B, C, f)$ 为 decode 单步时间。当前原生 timing 模型按频率进行非负拟合, 可简写为:

$$T_p(L, f) = a_f + b_f L + c_f L^2, \quad (1)$$

$$T_d(B, C, f) = u_f + v_f B + w_f BC. \quad (2)$$

其中线性项刻画主要工作量, BC 反映上下文相关访存; 代码内部的长度归一化已吸收入系数。历史兼容模型还支持 B^2 项和分段曲线, 实际查询由绑定的 profile 版本决定。功率同样按频率及 batch/context 建模, M 的功率在具备证据时直接采用混合服务窗口模型。

时间与功率必须对应同一观测区间。对长度为 H 的服务窗口, 其平均功率 $\bar{P}$ 与能量满足

$$E_{\text{window}} = \int_0^H P(t) dt = H\bar{P}. \quad (3)$$

原生 request-cycle 平均功率覆盖请求调度、计算及间隙, 不能直接乘仅包含 CUDA kernel 的 prefill 时间。相应地, 整池能耗由匹配的周期或布局窗口估计; 停车实例仍计静态功率。这样可以避免把不同口径的测量拼成虚假的节能收益。

传输、验证与查询。 基础 KV 模型采用固定开销加数据量除以带宽: $T_x(L) = x_0 + Lk/v$, 其中 k 是每 token 的 KV 字节数, v 是有效带宽。较新的原生组件也可按已测长度插值。然而, M/PD 路径的延迟差包含连接器与调度影响, 并非纯物理拷贝时间; 第2节进一步使用 P 首 token 到 D 首 token 的端点间隔描述真实交接。

拟合与参数选择使用 calibration/tuning 数据, 独立 holdout 用于检查预测误差与已选配置。发布时同时绑定允许的频率、长度、batch/context 和拓扑范围; 有界版本的越界查询返回缺少覆盖, 不能凭插值或理论带宽扩大适用域。规划器还检查静态状态、容量、传输和频率切换组件是否齐备。Profiler 因而提供的不只是一个预测值, 而是带有身份、测量口径和有效范围的决策依据; 决策记录绑定实际使用的 profile, 便于回查。

2 在线算法: 实例配对与 KV 交接

请求到达时, Router 固定它的 prefill 和 decode 归属, 实例内每轮 token 的组批仍由引擎 scheduler 完成。设请求输入为 L 个 token, 最多输出 K 个 token。M 路径选择同一个实例处理两阶段; PD 路径选择有序对 (p, d) , 由 p 处理 prompt, 由 d 继续生成。一个请求使用两个实例不意味着集群只有两个实例, P、D 池都可以具有多个副本。

2.1 候选路径与两个实例的选择

Router 首先读取当前已发布且允许接收新请求的角色表。P/D 配对要求两者是不同实例, 模型、TP、PP、池标识与 generation 一致, 且当前支持 PP=1; 现有 KV 连接器不提供跨 TP 的形

状重映射。由此得到所有兼容 P/D 对，而不是先独立选择 P 和 D 后再检查能否传输。

在兼容对存在时，输入超过阈值 τ 的请求优先进入 PD 候选分支，其余优先进入 M。没有可用 M 时，短请求也可以进入 PD；没有完整 P/D 对时，长请求仍可使用 M。启用压力策略时，阈值还受其最短 PD 输入约束。 τ 因而控制的是优先分支，不直接决定最终实例。

Router 为实例 i 维护两个代理负载： U_i 为已分配但尚未观察到首 token 的 prompt token 总数， N_i 为已分配且尚未结束的 decode 序列数。PD 派发时就增加 U_p 和 N_d ，使后续请求能看到尚在 prefill 的未来 D 负载。 U_i 不是逐 chunk 的精确剩余工作量， N_i 也不等于引擎当前运行的 batch。基础负载规则在 M 中按 (N_i, U_i) 选择，在兼容 PD 对中按 (U_p, N_d, U_d) 进行字典序比较。

当前标准入口默认启用 SLO 路由层。对优先分支中的全部候选，它先检查请求长度、准入状态和 KV 容量；PD 两端都按请求的 $L + K$ 预算进行代理侧准入记账，实际 KV block 仍由引擎分配。随后按各自当前控制频率查询 profile：P 排队时间由等待请求的 prefill 代价逐项累加，D 的代价结合已分配序列及其上下文上界估计。逐请求求和避免将多个短 prompt 合成一个超出测量域的虚构长 prompt。

2.2 将 KV 开销放入正确的延迟预算

PD 的代理协议先向 P 提交请求。P 完成 prefill 后生成并返回第一个 token，代理立即将它转发给客户端，再携带首 token、传输标识与 KV 相关元数据向 D 提交后续生成，剩余输出预算为 $K - 1$ 。连接器可以在 P 执行过程中分层提交 KV；不能把整个拷贝过程视为首 token 之后才开始。D 侧收到所需 KV 后继续解码。若 $K = 1$ ，预先转为仅在 P 生成的路径，不启动远程 KV 传输。

算法 1 普通 Router 的请求分配

输入： 请求 (L, K) 、角色表、profile、SLO

- 1: 根据阈值与兼容性生成优先候选集合
- 2: 若优先 PD，同时生成 M 备用集合
- 3: 删除长度、准入或 KV 容量不满足的候选
- 4: **for** 每个剩余候选 **do**
- 5: 估计排队、计算、交接与已有请求的预算
- 6: 标记 profile 覆盖及 TTFT/TPOT 可行性
- 7: **end for**
- 8: **if** 优先集中有满足 SLO 的候选 **then**
- 9: 选择其中预测 TTFT 最小者
- 10: **else if** 优先 PD 且存在满足 SLO 的 M **then**
- 11: 选择预测 TTFT 最小的 M
- 12: **else**
- 13: 沿用容量允许的原分支，并标记 SLO 未证实
- 14: 原分支也无容量时返回无路由
- 15: **end if**
- 16: 固定 (p, d) ，更新 U_p, N_d ，启动对应协议

设 Q_p 为 P 端排队时间， $s = T_d(B, C, f_D)$ 为 D 单步时间， h 为 P 首 token 到 D 首 token 的间隔。当 $K \geq 2$ 时，普通 Router 预测：

$$\text{TTFT}_{PD} = Q_p + T_p(L, f_P), \quad (4)$$

$$\text{TPOT}_{PD} = \frac{h + (K - 2)s}{K - 1}. \quad (5)$$

h 包括剩余 KV 交接、代理提交、D 排队/接收以及 D 的第一步，已测得该端点间隔时不能再加一次 s 。当 $K = 2$ ，TPOT 就等于整个交接间隔；输出较长时，该一次性成本才会被后续 token 分摊。这也解释了为何仅按输入长度判断 PD 是否有利会遗漏短输出的风险。M 的基础预测则为 $Q_m + T_p(L, f_M) + T_d(B, C, f_M)$ ；还要检查新增 prefill 是否使已有请求的首 token 或剩余输出预算超限。

端点模型缺失时，较长输出沿用 $h = \max(T_x(L) + s, h_{\min})$ 的风险近似， $h_{\min}$ 是配置的交接下限，不是测量证据。两 token 输出要求匹配请求形状、频率与批大小的端点覆盖，才可判定 PD 满足 SLO。服务期间记录 P 首 token、D 首 token 与后续输出，形成实际交接和进度观测；这些日志本身不会自动扩展离线模型的合格范围。

2.3 安全回退与代价边界

算法1优先选择原分支中留有 TTFT/TPOT 安全余量的最小预测 TTFT 路径；仅在 PD 分支无安全候选而 M 可行时向 M 溢出。M 保护也覆盖已有请求，避免把新长 prompt 转给一个看似空闲、实际剩余输出预算不足的实例。若预测缺失或全部超限，当前实现仍可保留原分支中容量允许的路径，并显式标记 SLO 未证实；因此模型过滤并不等价于所有获准请求都获得 SLO 保证。已派发请求保持原实例归属；已提交引擎且清理状态不明的异常请求，需收到原生清理确认才释放记账。

上述首间隔预算对应普通 Router。当前池级规划和跨 TP 的 ResidentRouter 仍保留将传输并入首步延迟的近似，尚未统一使用式(5)。ResidentRouter 可显式启用增量能量排序：先检查覆盖、SLO 和容量，再比较加入该请求后的增量焦耳数；任一保留候选缺少合格能量证据时，整个候选集合回退到时间评分。纯 KV 拷贝能耗没有独立证据时，不另造一个精确数值；服务窗口能耗与已验证的路径增量能量承担相应计费。

3 Adaptive Pool Configuration: 自适应池配置

请求调度处理单次到达，自适应配置则决定后续一段时间的服务能力。控制器根据近期到达率、输入输出长度及尚未完成的工作，联合选择 P、D、M 实例数、各角色频率、停车状态和 PD/M 分流阈值。以 c 表示配置， H 表示计划保持窗口，其目标为

$$c^* = \arg \min_{c \text{ feasible}} \{HP(c) + E(c_0, c)\}, \quad (6)$$

其中 $P(c)$ 包含工作与停车实例的功率， $E(c_0, c)$ 是转换额外能量的测量值或估计值。可行候选须满足带余量的 TTFT、TPOT、KV 容量和 profiler 覆盖要求，且保留仍有在途请求的服务分支。规划器对每个候选分别计入转换成本；当前配置可行时，只有收益超过相对阈值才切换。默认摊销窗口为 60 秒，标准 PDblend 策略将节省阈值设为 8%，均可被运行配置覆盖。转换成本优先采用绑定配置与 profile 的配对测量；严格模式排除缺少

证据的转换，普通模式回退到唤醒与调频估计，不将其标为实测值。

负载输入不仅包含新请求速率，也包含等待 prefill 的 token 数、剩余 decode 工作量和已经占用的 KV。长请求即使离开近期统计窗口，也继续计入在途快照。控制器默认每秒观察保护信号、每十秒进行普通规划；测量覆盖不足时，维持能够解释的当前配置或进入保守恢复路径，而非把无法预测的候选当成节能机会。

3.1 Frequency Control

频率控制在保持模型与 TP 拓扑不变条件下改变服务速率。规划器枚举 profiler 支持的离散频点，根据各阶段的负载、预测时延和功率选择 f_P, f_D, f_M 。P 与 D 可以使用不同频率，从而分别适应 prefill 与 decode 的资源需求；同一实例的全部 TP GPU 使用一致的目标频率。当前搜索是离散候选枚举，实际执行只修改状态发生变化的实例。

周期规划之外，启用预算感知保护器时，快速路径根据压力来源定位应调整的角色：PD 的 prefill 压力作用于 P，decode 压力作用于 D，混合路径压力作用于 M。其首级动作提高相关角色频率，持续压力再触发容量恢复；短暂的集体 token 间隙只允许一次有界调频探测。启用截止时间保护时，控制器还会在较高频点中寻找能够消除当前风险的最低频率，并为 M 维持频率下限，直到连续安全观察后释放。降频同时受保持时间、冷却窗口和多次预测确认约束，避免噪声引起反复切换。

物理执行以 GPU 标识加互斥锁，防止不同控制动作同时写入同一张卡；阻塞的硬件调用放入工作线程，使正在加载的实例不会阻塞其他实例的紧急调频。

3.2 Role Change

角色控制改变各阶段获得的实例数量。规划器在满足总实例预算和服务约束的布局中选择 P/D/M 比例，并可将剩余实例置为驻留低功耗状态或停止进程。将目标数量映射到具体实例时，首先保留已有角色，再结合驻留状态与负载安排变更，以减少不必要的唤醒和角色重分配。

算法 2 自适应池配置的控制流程

输入： 当前配置、profile、历史请求与在途工作

- 1: 更新到达率、长度分布及保留归属的 backlog
 - 2: 快环观察 TTFT、TPOT 与输出进度
 - 3: **if** 出现风险 **then**
 - 4: 提频；持续容量压力下恢复实例，保护保持期
 - 5: **end if**
 - 6: **if** 到达周期或触发重规划，且允许优化 **then**
 - 7: 枚举角色、频率、停车状态与分流阈值
 - 8: 联合 TP 模式在外层枚举两池流量权重
 - 9: 按覆盖、容量和 SLO 删除不可行候选
 - 10: 最小化窗口能量加转换成本
 - 11: 检查节省阈值、保持时间与降配确认
 - 12: 保留在途归属，执行所需物理变更
 - 13: 联合模式等待各池就绪，再发布流量权重
 - 14: **end if**
 - 15: 记录实际状态与分流数，反馈到下一周期
-

当前实现区分路由角色调整与释放资源。P、D、M 活跃角色之间的转换仅更新后续请求的路由角色及频率；已进入系统的请求继续绑定原实例，不迁移在途 KV。因此，角色改变可以渐进生效，过渡期间实例仍需完成先前接收的工作。规划时的 backlog 保留其原有分支归属，不能因为目标角色改变而删除存量负载。

当实例进入停车或停止状态时，控制器先关闭新请求准入，再等待代理侧 prefill 与 decode 工作清零，并在接入原生控制时核查全部 rank 的队列、KV 块及待完成传输均已释放，然后执行降功耗或停止。唤醒则完成启动或恢复、就绪确认、调频和原生准入恢复后才向路由器发布。多个实例的动作可以并行，但任何失败都阻止受影响实例继续接收请求。由此，轻量角色变化保留服务连续性，资源回收具有明确的排空边界。

标准策略保留至少 $\min(4, N)$ 个 M，其中 N 为池内实例数；因此默认候选不等同于任意纯 PD 布局。降低此下限需要匹配当前模型、拓扑、SLO 和负载域的验收记录，具体入口还约束频率并预

留可恢复的 M 容量。该约束补足排队近似对复杂混合干扰的描述，避免过度缩容。

3.3 TP Weight Change

当前 TP 权重改变指两个常驻异构 TP 池之间的请求份额。两个池使用不同的 TP 大小和不重叠的 GPU，各自绑定独立 profile；在线调整保持既有模型分片布局。外层规划枚举两池权重 w 与 $1 - w$ ，内层分别求解角色和频率配置，并按式 (6) 汇总两池能量。默认权重步长为 0.1，候选还包括当前目标份额和上一周期的实际份额。

对于每个权重，未来到达率按份额拆分，而已进入系统的请求仍计入原池 backlog。即使某池权重降为零，其全部驻留副本和空闲 GPU 仍参与功耗核算，防止将备用容量视为免费资源。协调器仅执行两池均满足覆盖、容量和 SLO 的候选；当前联合配置仍可行时，新目标还须达到节省阈值。全部角色、频率和准入状态就绪后才发布新权重。

联合规划还要求可用的实测 decode 与混合窗口功率。保护器处于扩容或保持阶段时暂缓新的能量优化；若任何池执行失败，保持原目标权重并暂停后续优化，等待真实状态恢复。算法2概括普通控制和联合 TP 控制的公共流程；两者共享快慢环思想，并不要求三类变更按固定顺序逐一发生。

在线分流采用份额欠额：若已分配 A 个请求、池 j 获得 a_j 个，则在当前可行池中选择

$$j^* = \arg \max_j \{w_j(A + 1) - a_j\}. \quad (7)$$

目标池不可行时可使用另一可行池，因此份额是控制目标而非强制拒绝条件。实际分配数反馈到下一周期，使联合规划能观察容量约束导致的偏离。此机制不要求跨 TP 搬移在途请求；在线模型权重重分片入口目前仍因缺少经过 GPU 验证的原生事务后端而关闭。